

Clarity Is Not Assumed: Understanding LLM-Based Code Generation under Ambiguous Requirements

Di Yang
East China Normal University
Shanghai Innovation Institute
Shanghai, China
diyang@stu.ecnu.edu.cn

Xinou Xie
East China Normal University
Shanghai, China
51275902162@stu.ecnu.edu.cn

Xiuwen Yang
East China Normal University
Shanghai, China
52295900006@stu.ecnu.edu.cn

Ming Hu
East China Normal University
Shanghai, China
mhu@sei.ecnu.edu.cn

Yihao Huang
East China Normal University
Shanghai, China
huangyihao@sei.ecnu.edu.cn

Yueling Zhang
East China Normal University
Shanghai, China
ylzhang@sei.ecnu.edu.cn

Weikai Miao
East China Normal University
Shanghai, China
wkmiao@sei.ecnu.edu.cn

Ting Su
East China Normal University
Shanghai, China
tsu@sei.ecnu.edu.cn

Chengcheng Wan*
East China Normal University
Shanghai Innovation Institute
Shanghai, China
ccwan@sei.ecnu.edu.cn

Geguang Pu
East China Normal University
Shanghai, China
ggpu@sei.ecnu.edu.cn

Abstract

Software requirement ambiguity is ubiquitous in real-world development, stemming from the inherent imprecision of natural language and the varying interpretations of stakeholders. While Large Language Models (LLMs) have demonstrated impressive capabilities in generating code from precise specifications, such ambiguity poses a significant obstacle to reliable automated code generation. Existing benchmarks typically assume clear and unambiguous requirements, leaving an empirical gap in understanding how LLMs behave when faced with the inherent uncertainty of real-world software requirements.

In this paper, we introduce **Orchid**, a code generation benchmark specifically designed with ambiguous requirements. It comprises 1,304 function-level tasks covering four distinct types of ambiguity: *lexical*, *syntactic*, *semantic*, and *vagueness*. Leveraging this dataset, we conduct the first systematic empirical study to evaluate the impact of requirement ambiguity on LLM-based code generation. Our results demonstrate that ambiguity consistently degrades the performance of all evaluated LLMs, with the most pronounced negative effects observed in highly advanced models. Furthermore, we observe that LLMs frequently produce functionally divergent implementations for the same ambiguous requirement and lack the capability to identify or resolve such ambiguity autonomously. These findings reveal a significant performance gap between clear and ambiguous requirements, underscoring the urgent need for ambiguity-aware techniques in the next generation of automated software engineering tools. The Orchid benchmark is publicly available at https://huggingface.co/datasets/SII-YDD/Orchid.

CCS Concepts

• **General and reference** → **Empirical studies**; • **Software and its engineering** → *Source code generation.*

Keywords

Automated Code Generation, Requirement Ambiguity, Empirical Study, Large Language Models

ACM Reference Format:
Di Yang, Xinou Xie, Xiuwen Yang, Ming Hu, Yihao Huang, Yueling Zhang, Weikai Miao, Ting Su, Chengcheng Wan, and Geguang Pu. 2026. Clarity Is Not Assumed: Understanding LLM-Based Code Generation under Ambiguous Requirements. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26), October 12–16, 2026, Munich, Germany.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3832783.3834403

1 Introduction

As emphasized by Brooks [8], determining precisely what to build remains one of the most challenging aspects of software development. This challenge lies at the core of Requirements Engineering (RE), which forms the foundation of the software development life cycle. In practice, requirements are predominantly documented

*Chengcheng Wan is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
ASE '26, Munich, Germany
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
https://doi.org/10.1145/3832783.3834403

in natural language, which is inherently imprecise and prone to ambiguity [5, 6, 15]. Such ambiguity, where a single description corresponds to multiple conflicting interpretations, is prevalent in real-world development due to limited communication and the varying expertise of stakeholders [19, 21].

While human developers can mitigate these uncertainties through iterative clarification, Large Language Model (LLM) based code generation faces a critical conflict. LLM-based solutions are typically forced into determinism, which collapses inherent linguistic uncertainty into a single, executable implementation without explicit clarification. As illustrated in Figure 1, an ambiguous requirement using the phrase *"filtered by"* results in LLMs generating functionally distinct code snippets. The ambiguity leads to two divergent interpretations: Implementation A retaining items above the threshold (`x > threshold`); Implementation B keeping only those below it (`x <= threshold`). This forced determinism compels models to make unwarranted assumptions, posing a significant barrier to reliable code generation.

Despite the rapid progress in LLM-based code generation, widely used function-level benchmarks generally assume that each natural-language requirement has a single intended interpretation, largely excluding linguistic uncertainty from evaluation [31, 42]. In parallel, the requirements engineering (RE) community has studied ambiguity from a human-centric perspective [17, 20]. However, how code LLMs behave when confronted with inherently ambiguous requirements remains underexplored, limiting our understanding of their reliability in realistic software development settings. This gap stems from a fundamental limitation in existing evaluation paradigms: ambiguity is neither explicitly modeled nor systematically controlled. As a result, prior studies are unable to disentangle whether failures arise from model deficiencies or from inherent uncertainty in the input specification.

To address this gap, we introduce **Orchid**, a function-level benchmark designed to evaluate the impact of requirement ambiguity on code generation. Orchid consists of 1,304 tasks and 5,216 ambiguous requirement variants, covering four types of ambiguity: *lexical*, *syntactic*, *semantic*, and *vagueness*. These types represent different sources of interpretive uncertainty, allowing us to assess how varying linguistic factors affect model behavior. The benchmark is constructed using a semi-automated pipeline that follows a general, reusable method for ambiguity data construction. This method is based on a multi-agent framework designed for ambiguity injection, ensuring both scalability and high-quality output. The process begins with the Injection Agent, which introduces ambiguity into each clean functional requirement based on predefined ambiguity types. After ambiguity is injected, the Judge Agent evaluates the variants to ensure that the injected ambiguity is contextually plausible and retains the original functional intent. Finally, the Explain Agent provides concise explanations of the plausible interpretations for each ambiguous requirement, clarifying the effects of the ambiguity. To ensure the quality of the manually curated core, experts conducted item-level validation of all 1,312 variants in Orchid-HEval and Orchid-BCB, requiring over 246 person-hours.

To delve into the challenges posed by requirement ambiguity, we conduct a comprehensive empirical study using Orchid, focusing on three critical dimensions of LLM behavior. First, we investigate the performance impact of ambiguity (**RQ1: How does ambiguity impact LLM performance?**). Our analysis reveals a pervasive and substantial degradation in generation quality across all evaluated models. Even state-of-the-art models, such as GPT-4, exhibit a performance drop exceeding 30% when confronted with ambiguous specifications, suggesting that current benchmarks significantly overestimate the effectiveness of LLMs in real-world, "noisy" software engineering scenarios. Second, we assess the functional consistency of LLMs under uncertainty (**RQ2: How consistent are LLMs in generating functional code under ambiguity?**). Beyond mere correctness, we find that ambiguity undermines the reliability of code generation by triggering functional divergence. Models frequently produce multiple, mutually incompatible implementations for the same ambiguous prompt across different trials. This lack of determinism indicates that LLMs struggle to maintain a stable internal representation when requirements are not strictly bounded. Third, we evaluate the models' intrinsic capability to recognize and resolve ambiguity (**RQ3: How well can LLMs recognize and resolve ambiguities?**). While LLMs demonstrate a surprisingly high recall in flagging potential ambiguities, they suffer from overprediction and uncertainty. More importantly, they consistently fail to precisely localize the source of ambiguity or provide valid resolutions, highlighting a fundamental gap between detecting a problem and understanding its logic.

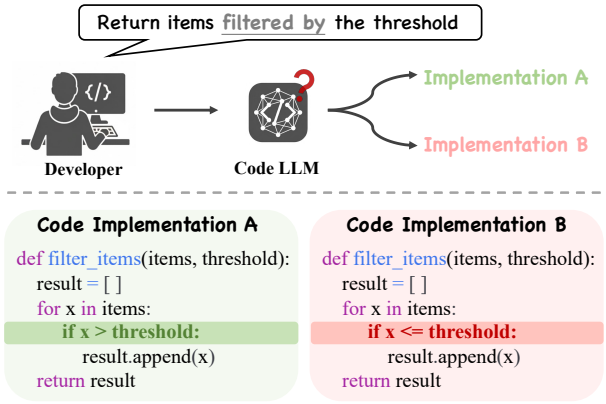


Figure 1: An ambiguous requirement results in LLMs generating functionally distinct code snippets. Here, *"filtered by"* is implemented in two different ways: (A) retaining items above the threshold; and (B) keeping only those below it.

These findings underscore that ambiguity is not merely a data-level noise but a structural bottleneck that compromises the trustworthiness of automated code generation. In summary, this paper makes the following contributions:

- **Ambiguity Benchmark:** We constructed Orchid, a benchmark of 1,304 tasks and 5,216 requirements spanning four ambiguity types, enabling evaluation of LLMs under ambiguity.
- **Empirical Impact Study:** We systematically quantify the impact of ambiguity on LLM performance and functional consistency across multiple models.
- **LLM Behavior Analysis:** We characterize how LLMs handle ambiguous requirements, focusing on their ability to recognize ambiguity and their limitations in localizing and resolving it.

Table 1: Code Generation Benchmarks.

Benchmark	Time	Granularity	#Tasks	Construction	Language	Req. Characteristics[2]
CoNaLa [41]	2018	Statement-level	500	Semi-automated	Python	Code-aligned
HumanEval [12]	2021	Function-level	164	Manual	Python	Concise
MBPP [4]	2021	Function-level	974	Manual	Python	Concise
MBXP [3]	2022	Function-level	974	Manual	Multi-language	Diverse
Multi-HumanEval [3]	2022	Function-level	164	Manual	Multi-language	Concise
DS-1000 [29]	2022	Statement-level	1000	Semi-automated	Python	Code-aligned
HumanEval+ [31]	2023	Function-level	164	Automated	Python	Concise
SWE-Bench [28]	2023	Pull Request-level	2294	Semi-automated	Python	Contextual
CRUXEval [23]	2024	Function-level	800	Semi-automated	Python	Multi-step
ClassEval [13]	2024	Class-level	100	Manual	Python	Structured
Safim [22]	2024	Statement-level	17720	Semi-automated	Multi-language	Code-aligned
CanItEdit [11]	2024	Function-level	105	Manual	Python	Instructional
LiveCodeBench [26]	2024	Function-level	1055[1]	Semi-automated	Python	Concise
BigCodeBench [42]	2024	Function-level	1140	Semi-automated	Python	Concise
Orchid (Ours)	**2026**	**Function-level**	**1304**	**Semi-automated**	**Python**	**Ambiguity**

[1] As of release_v6 (Apr 2025).
[2] Characteristics of code generation requirements.

2 Background

2.1 LLM-Based Code Generation

Large language models (LLMs) have significantly improved code generation from natural language requirements, enabling the production of syntactically correct and semantically meaningful programs [12, 33, 38]. These models achieve strong performance when requirements are clearly specified, where the intended functionality can be directly inferred from the input.

Code generation relies on the model's interpretation of natural language. When a requirement admits multiple plausible interpretations, the generation process becomes underdetermined. In such cases, the model must resolve ambiguity implicitly and produce a single implementation without external clarification. Different interpretations can therefore lead to functionally different outputs. Despite the importance of this problem, the impact of requirement ambiguity on code generation remains insufficiently understood.

2.2 Code Generation Benchmarks

Code generation benchmarks provide standardized datasets for evaluating a model's ability to translate natural language requirements into executable programs. Existing benchmarks cover different levels of granularity, including statement-level tasks that focus on syntactic correctness [22, 29, 41], function-level tasks that evaluate semantic correctness and reasoning [3, 4, 11, 12, 23, 26, 31, 42], and higher-level tasks that involve classes or repositories [13, 28].

As shown in Table 1, widely used benchmarks such as HumanEval [12], MBPP [4], and BigCodeBench [42] adopt concise and well-defined requirements. These datasets are constructed through manual curation or semi-automated pipelines with human validation, which ensures clarity and consistency in task descriptions.

Most existing benchmarks share a common assumption that each requirement has a single intended interpretation. This assumption simplifies evaluation and enables consistent measurement of functional correctness. However, real-world requirements often contain ambiguity, where multiple interpretations are possible. As a result, current benchmarks mainly evaluate model performance under idealized conditions and do not capture model behavior under ambiguous requirements.

2.3 Ambiguity in Software Requirements

Ambiguity in software requirements arises when a specification allows multiple valid interpretations [6, 36]. It can result from insufficient information, such as under-specification or vagueness, as well as linguistic factors including lexical, semantic, and syntactic ambiguity. Prior work in requirements engineering has studied ambiguity detection and mitigation, with a primary focus on supporting human developers [17, 20].

In LLM-based code generation, ambiguity introduces a fundamental challenge. Human developers can iteratively refine and clarify requirements [19], while LLMs are typically required to produce a single output given the input. This mismatch leads to inconsistent or incorrect implementations when different interpretations are possible. In addition, different runs or different models resolve ambiguity in different ways, resulting in functional divergence.

3 Requirement Ambiguity

As shown in Table 2, our paper focuses on function-level requirements and considers four types of ambiguity — lexical, semantic, syntactic, and vagueness, following a study of natural language software requirements [36]. We omit pragmatic ambiguity, as it involves implied intentions or assumptions that rarely appear in function-level requirements. We also omit language errors and generality problems, as they are less relevant to code generation.

Not all ambiguity of requirements could lead to confusion when generating the code. We regard the ambiguity as taking effect if it could be interpreted in multiple plausible ways, each corresponding to a functionally different implementation. Formally, an NL

Table 2: Types of Ambiguities in Function-Level Code Generation [36].

Types	Definition
Lexical	A word in the requirement has multiple distinct meanings, causing different interpretations.
Syntactic	A requirement's grammatical structure has multiple distinct meanings, causing different interpretations.
Semantic	A phrase or sentence in the requirement has multiple distinct meanings, causing different interpretations.
Vagueness	A requirement that omits necessary details has multiple distinct meanings, causing different interpretations.

requirement R is ambiguous for code generation only when

$$\exists I, I' \in \mathbb{I}(R), s.t.\ \mathrm{F}_I \neq \mathrm{F}_{I'}, \tag{1}$$

$$\mathrm{F}_I \neq \mathrm{F}_{I'} \iff \exists x, s.t.\ \mathrm{F}_I(x) \neq \mathrm{F}_{I'}(x) \tag{2}$$

where $\mathbb{I}(R)$ is the set of all plausible interpretations of R, F_I is the functionality of interpretation I, and $\mathrm{F}_I(x)$ is the expected output when given an input x.

3.1 Lexical Ambiguity

```
from typing import List
def filter_by_substring(strings: List[str], substring:
    str) -> List[str]:
    Filter an input list of strings only for ones that
    contain given pattern.
```

(a) Lexical ambiguity in Orchid-HEval task #7.

```
# Interpretation A
def filter_by_substring(strings: List[str], substring:
    str) -> List[str]:
    return [s for s in strings if substring in s]

# Interpretation B
def filter_by_substring(strings: List[str], substring:
    str) -> List[str]:
    regex = re.compile(pattern)
    return [s for s in strings if regex.search(s)]
```

(b) Divergent implementations.

Figure 2: A lexical ambiguity example from Orchid-HEval task #7, where 'pattern' is interpreted in multiple ways.

Lexical ambiguity appears when a word in the requirement has multiple meanings that are seemingly feasible in its context. It often arises from polysemous words with related senses or from vague terms with an undefined scope. In software requirements, this ambiguity is harmful because the interpretation of a single word can even alter the entire implementation.

Figure 2a illustrates an example of lexical ambiguity in the requirement. The requirement uses the term "pattern," which can be interpreted either as a literal substring or as a more general pattern such as a regular expression, leading to multiple possible implementations. Specifically, this ambiguity may lead to two different implementations. Implementation A (Figure 2b) checks if each string contains the input as a literal substring and returns those that do. Implementation B (Figure 2b) treats the input as a regular expression pattern, using regex matching to return all strings that satisfy the pattern. For test case `strings = ["aaa", "aa", "a", "b"]`, `substring = "a*"`, implementation A returns an empty list `[]`, while implementation B returns `["aaa", "aa", "a"]`.

3.2 Syntactic Ambiguity

```
def unique(l: list):
    Sort the list and return the unique elements in it.
```

(a) Syntactic ambiguity in Orchid-HEval task #34.

```
# Interpretation A
def unique(l: list):
    return sorted(set(l))

# Interpretation B
def unique(l: list):
    sorted_l = sorted(l)
    return list(set(l))
```

(b) Divergent implementations.

Figure 3: A syntactic ambiguity example from Orchid-HEval task #34, where the pronoun "it" has an unclear reference.

Syntactic ambiguity occurs when the grammatical structure of a requirement sentence allows multiple valid interpretations, creating uncertainty in how its components are organized and related. Resolving syntactic ambiguity through context alone is often challenging, particularly for complex or lengthy sentences, where multiple parses may remain plausible despite rich contextual cues. Furthermore, syntactic ambiguity can fundamentally change the requirements' logic and behavior by affecting interpretations of action order, conditional scopes, and other critical aspects, thus causing risks to the correct implementation.

Figure 3a illustrates an example of syntactic ambiguity. The requirement involves returning the unique elements of a list, but the pronoun "it" can refer either to the sorted list or to the original unsorted list. This syntactic ambiguity results in two implementations. Implementation A (Figure 3b) returns the unique elements in sorted order by directly applying a set to the list and then sorting the result. Implementation B (Figure 3b) first sorts the list (though this sorted list is unused), then returns the unique elements of the original list without preserving any particular order. For test case `l = [4, 3, 3, 1, 2, 5, 1]`, implementation A returns `[1, 2, 3, 4, 5]`, while implementation B returns `[4, 3, 1, 2, 5]`.

3.3 Semantic Ambiguity

Semantic ambiguity occurs when a phrase or sentence contains expressions that allow multiple plausible interpretations within the context of the requirement. This means that the same part of

```
def remove_duplicates(numbers: List[int]) -> List[int]:
    From a list of integers, remove duplicate occurrences,
    preserving the initial order of the remaining
     elements.
```

(a) Semantic ambiguity in Orchid-HEval task #26.

```
# Interpretation A
def remove_duplicates(numbers: List[int]) -> List[int]:
    result = []
    for x in numbers:
        if numbers.count(x) == 1:
            result.append(x)
    return result

# Interpretation B
def remove_duplicates(numbers: List[int]) -> List[int]:
    seen = set()
    result = []
    for x in numbers:
        if x not in seen:
            seen.add(x)
            result.append(x)
    return result
```

(b) Divergent implementations.

Figure 4: A semantic ambiguity from Orchid-HEval task #26, where 'duplicate occurrences' is interpreted differently.

the requirement can be reasonably understood in different ways, potentially leading to varied implementations. Compared to lexical ambiguity, resolving this ambiguity requires broader contextual information and deeper reasoning.

Figure 4a illustrates an example of semantic ambiguity. The requirement instructs the removal of duplicate occurrences in a list, which allows for two plausible interpretations: Implementation A (Figure 4b) filters out all repeated elements, returning only those with a single occurrence. Implementation B (Figure 4b) preserves the first occurrence of each integer, removing subsequent duplicates while maintaining the original order. For test case `numbers = [1, 2, 2, 3, 4, 4, 5]`, implementation A returns `[1, 3, 5]`, while implementation B returns `[1, 2, 3, 4, 5]`.

3.4 Vagueness Ambiguity

Vagueness ambiguity arises when a requirement includes expressions that omit necessary details or lack sufficient specificity, leading to incomplete information. This deficiency permits multiple plausible interpretations within the requirement's context, resulting in varied understandings and implementations. Addressing vagueness ambiguity typically involves supplementing missing information or clarifying constraints.

Figure 5a illustrates an example of vagueness ambiguity. The requirement does not specify whether the product should be computed over all digits or only a subset, leaving the interpretation of which digits to include unclear. This lack of specificity permits two implementations: Implementation A (Figure 5b) multiplies only the odd digits, whereas Implementation B (Figure 5b) multiplies all digits regardless of parity. For test case `n = 3526`, implementation A returns `15`, while implementation B returns `180`.

```
def digits(n: int) -> int:
    Given a positive integer n, return the product of the
    digits.
```

(a) Vagueness ambiguity in Orchid-HEval task #131.

```
# Interpretation A
def digits(n):
    odd_digits = []
    for d in str(n):
        if int(d) % 2 != 0: odd_digits.append(int(d))
    return math.prod(odd_digits) if odd_digits else 0

# Interpretation B
def digits(n):
    digits = [int(d) for d in str(n)]
    return math.prod(digits)
```

(b) Divergent implementations.

Figure 5: A vagueness ambiguity example from Orchid-HEval task #131, where the term "digits" is unspecified.

4 Orchid Construction

4.1 Methodology

Orchid is constructed using the semi-automated human-in-the-loop pipeline shown in Figure 6. The pipeline comprises three stages: requirement extraction, requirement rewriting, and human curation. Before rewriting, we specify the target ambiguity type for each generation request. For each source requirement, we produce one *lexical*, one *syntactic*, one *semantic*, and one *vagueness* variant. We use the source programming task as the unit of construction. Each original requirement and its four variants form a matched set, allowing model behavior to be compared across ambiguity conditions while holding the source task constant.

4.1.1 Requirement Extraction. To prepare inputs for ambiguity rewriting, we first extract function-level requirements from existing datasets (green part in Figure 6). We use a rule-based parser to extract the natural language, and remove implementation-specific elements (*e.g.*, input/output examples or code snippets), as clear requirements for rewriting.

4.1.2 Requirement Rewriting. We inject ambiguity into each source requirement using the multi-agent framework shown in Figure 6. The framework consists of an *Ambiguity Injection Agent*, an *Ambiguity Judge Agent*, and an *Ambiguity Explanation Agent*, all implemented using DeepSeek-V3 [30]. The rewriting process aims to introduce a contextually plausible ambiguity while preserving the original functional intent.

First, the *Ambiguity Injection Agent* rewrites each requirement according to a predefined target type. The target type is provided as input rather than inferred from the generated requirement. Guided by its definition and representative in-context examples [9], the agent introduces one representative instance of the designated ambiguity while avoiding the other ambiguity types.

Next, the *Ambiguity Judge Agent* evaluates each candidate according to three criteria: type specificity, implementation impact, and naturalness. Based on these assessments, the judge determines whether the candidate passes machine screening. If a candidate

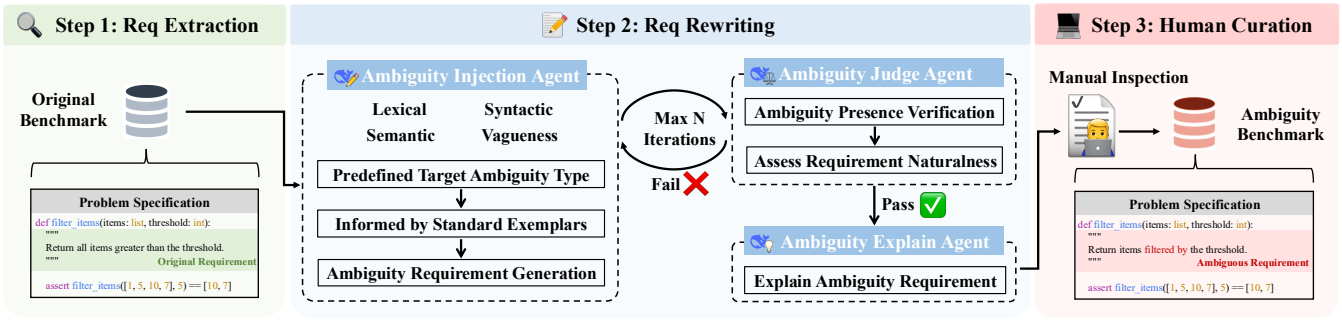


Figure 6: Overview of Orchid Construction Process.

Subset	# (Ori / Amb)	Original	Lexical	Syntactic	Semantic	Vagueness	Avg Amb	Avg Δ (%)
Token Length								
Orchid-HEval	164 / 656	143.47	140.23	140.12	140.78	135.87	139.25	-2.94
Orchid-BCB	164 / 656	181.26	178.18	179.16	185.09	176.90	179.83	-0.79
Orchid-BCB-Expand	976 / 3904	175.84	173.49	174.25	180.27	171.14	174.79	-0.60
Perplexity								
Orchid-HEval	164 / 656	30.55	37.39	37.04	32.81	36.72	35.99	+17.78
Orchid-BCB	164 / 656	23.17	24.11	24.23	24.67	23.83	24.21	+4.49
Orchid-BCB-Expand	976 / 3904	24.99	26.51	25.93	26.51	25.92	26.22	+4.92

Note: Avg Δ = (Avg Amb − Original) / Original ×100%.

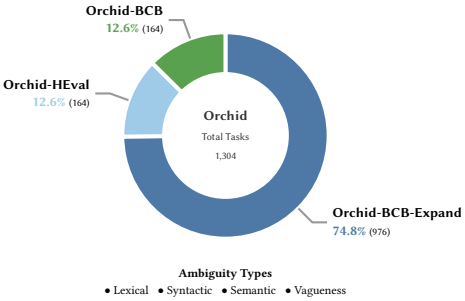


Figure 7: Orchid Benchmark Statistics. The chart displays the distribution of data sources and lists the covered ambiguity types, while the table details token length and perplexity for original and ambiguous requirements.

fails, the injection and evaluation steps are repeated for up to five iterations. If no candidate passes within this limit, the highest-rated candidate is forwarded to human curation rather than accepted automatically.

Finally, the *Ambiguity Explanation Agent* identifies the ambiguous expression in the selected candidate and summarizes its plausible interpretations in a consistent format. The explanation is retained with the candidate to support subsequent human inspection and downstream analysis.

4.1.3 Human Curation. As illustrated in Figure 6, human curation constitutes the final stage of our construction pipeline for benchmark subsets designated for item-level validation. During this stage, the generated variants are partitioned among the curators, and each curator compares every assigned rewritten requirement with its corresponding original requirement.

Each variant is assessed against three criteria. First, the ambiguity introduced by the rewritten wording must be consistent with the intended ambiguity type. Second, the original intended behavior must remain a plausible interpretation of the rewritten requirement, and the underlying programming task must remain unchanged apart from the targeted ambiguity. Third, the rewritten requirement must support at least two plausible interpretations that could lead to functionally different implementations, rather than merely different wording or implementation styles.

Variants that fail any criterion are returned for requirement revision or ambiguity reinjection, as appropriate. Revised variants are inspected again using the same criteria and retained only after satisfying all three.

4.2 Orchid Instantiation

We instantiated the preceding construction procedure using two established function-level code-generation benchmarks: HumanEval+ [31] and BigCodeBench [42]. Orchid-HEval includes all 164 tasks from HumanEval+. To construct a size-matched core subset, we selected the first 164 tasks from BigCodeBench to form Orchid-BCB. For each task in these two subsets, the requirement-rewriting procedure generated four ambiguity variants.

Together, Orchid-HEval and Orchid-BCB constitute the manually curated core of Orchid, comprising 328 source tasks and 1,312 ambiguity variants. All 1,312 variants underwent the item-level curation protocol described in Section 4.1.3. The inspection and subsequent revision of these variants required over 246 person-hours.

To extend the scope of our evaluation beyond the 328-task manually curated core, we applied the same requirement-rewriting procedure to the remaining 976 BigCodeBench tasks. This produced 3,904 additional variants, collectively referred to as Orchid-BCB-Expand, enabling a larger-scale evaluation of model behavior under ambiguous requirements. Because this extension did not undergo the item-level manual curation applied to Orchid-HEval and Orchid-BCB, we report its results separately to make the difference in validation coverage explicit. Overall, Orchid comprises 1,304 source tasks and 5,216 ambiguity variants across the three subsets.

Table 3: Selected LLMs.

Category	Models	Size	Publisher	Open Source	Release
General	GPT-4 [34]	N/A	OpenAI	No	Mar 2023
	DeepSeek-V3 [30]	671B	DeepSeek	Yes	Dec 2024
	Claude-3.5 [1]	N/A	Anthropic	No	Jun 2024
Code	CodeLlama-34B [35]	34B	Meta	Yes	Aug 2023
	Qwen-2.5-Coder [25]	32B	Alibaba	Yes	Sep 2024
Reasoning	DeepSeek-R1 [24]	671B	DeepSeek	Yes	Jan 2025

4.3 Benchmark Statistics

As summarized in Figure 7, Orchid covers 1,304 tasks and 5,216 ambiguous requirements across lexical, syntactic, semantic, and vagueness. It comprises Orchid-HEval and Orchid-BCB, with 164 tasks each, and Orchid-BCB-Expand with 976.

Ambiguity causes only minor variations in token-level requirement length. In Orchid-HEval, Orchid-BCB, and Orchid-BCB-Expand, the average token length decreases by 2.94%, 0.79%, and 0.60%, respectively. These variations indicate that the overall structural characteristics of the tasks are preserved. Among ambiguity types, vagueness variants tend to be slightly shorter, whereas semantic variants are relatively longer.

Ambiguity also increases linguistic uncertainty in requirements. In all subsets, ambiguous requirements consistently exhibit higher perplexity than originals, with an average increase of 17.78% in Orchid-HEval, 4.49% in Orchid-BCB, and 4.92% in Orchid-BCB-Expand. Lexical and syntactic variants generally contribute more significantly to the perplexity increase, while semantic and vagueness variants lead to smaller, though still notable, increases.

5 Benchmarking Analysis

5.1 Experimental Setup

LLM Selection. We adopt a series-representative strategy to efficiently cover major model families while balancing capability and computational cost. As shown in Table 3, we select six LLMs, covering three categories of general-purpose, code-specialized, and reasoning models across diverse parameter sizes.

Evaluation Protocol. We use the source benchmark task as the unit of analysis. For each task, we evaluate the original requirement and its four corresponding variants, one for each ambiguity type. For each combination of model and benchmark, the original requirement and its variants use the same prompt wrapper, generation configuration, and source-task tests. We generate $n = 5$ completions for every model and requirement pair. Therefore, each ambiguity variant is compared with its corresponding original requirement over the same set of source tasks, rather than with an unrelated set of clear requirements. For each ambiguity type, we define $\Delta = \text{Pass@}k_{\text{ambiguous}} - \text{Pass@}k_{\text{original}}$, where a negative value indicates performance degradation.

LLM Settings. We retain the benchmark-specific prompt formats to preserve comparability with the original evaluations. For each model and benchmark, the original requirement and its four ambiguity variants are evaluated under identical generation settings, ensuring a controlled comparison of requirement wording.

Test Suites. For Orchid-HEval, we use an EvalPlus-based evaluation harness with the HumanEval+ tests [31]. For Orchid-BCB and Orchid-BCB-Expand, we use the tests associated with the corresponding BigCodeBench source tasks [42]. The same tests and execution limits are applied to each original requirement and its corresponding variants. Syntax errors, runtime exceptions, and timeouts are counted as failures.

Evaluation Metrics. We use the following metrics:

- *Pass@k*: The estimated probability that at least one of the k generated code samples passes all unit tests: $\text{Pass@}k = 1 - \binom{n-c}{k} / \binom{n}{k}$, where c is the number of correct samples among $n = 5$ generations. We report the average across tasks for $k \in \{1, 3, 5\}$.
- *Conflict Rate*: The proportion of response pairs whose test-outcome vectors differ on at least one test: $\text{ConflictRate} = C / \binom{r}{2}$, where C is the number of conflicting pairs among the r responses. We compute the conflict rate for each task and average across tasks.
- *Ambiguity Recognition*: We assess whether a model can recognize, localize, and address requirement ambiguity using four ascending levels: (i) *Unaware*, where the model does not recognize ambiguity; (ii) *Detection*, where it recognizes ambiguity but does not identify its source; (iii) *Localization*, where it correctly identifies the ambiguous segment; and (iv) *Tackling*, where it provides plausible interpretations or clarification options. Detection, Localization, and Tackling are treated as positive ambiguity detections when computing precision and recall.

5.2 RQ1: Performance Impact

Table 4 summarizes the Pass@k results. Overall, ambiguity poses a pervasive challenge to reliable code generation. It consistently degrades generation quality across all evaluated models, reducing Pass@1 accuracy by an average of 7.22 percentage points, with the largest observed decline reaching 32.35 points.

Notably, strong performance on clear requirements does not necessarily translate into stable behavior under ambiguous inputs. Orchid is effective in revealing such latent capability gaps that are not captured by standard benchmarks. For example, although GPT-4 achieves top-tier baseline performance, its accuracy drops by more than 28 percentage points on Orchid-BCB under ambiguous requirements. In contrast, open-source models such as Qwen-2.5-Coder exhibit relatively higher stability, with performance degradation limited to approximately 8 percentage points.

In addition, Orchid reveals fine-grained intra-model sensitivity to different types of ambiguity. For instance, Claude-3.5 is highly affected by syntactic ambiguity, with a performance drop of 31.10 percentage points, while its performance remains relatively stable under semantic ambiguity, with a decline of only 2.44 percentage points. These results indicate that ambiguity affects models in a non-uniform manner and interacts with both model-specific characteristics and ambiguity types. Overall, these findings highlight the necessity of Orchid for a comprehensive evaluation of LLMs under realistic and ambiguous requirements.

> **Finding 1:** High performance on clear requirements does not guarantee stability under ambiguity. On average, models suffer a relative performance decline of 16.25% in Pass@1 accuracy.

Table 4: Pass@k (%) on original and paired changes. Each original requirement and its four ambiguity variants are evaluated as a paired group using the same model configuration and test suite.

Models	Orchid-HEval					Orchid-BCB					Orchid-BCB-Expand				
	Orig.	Δ Lex	Δ Syn	Δ Sem	Δ Vag	**Orig.**	Δ Lex	Δ Syn	Δ Sem	Δ Vag	**Orig.**	Δ Lex	Δ Syn	Δ Sem	Δ Vag
Pass@1 (%)															
CodeLlama	**33.66**	-4.39	-2.93	-2.32	-7.07	**6.22**	-1.34	-1.34	-1.59	-0.49	**5.72**	-0.84	-0.93	-1.42	-0.58
Qwen-2.5-Coder	**69.15**	-6.59	-3.78	-2.93	-9.15	**42.80**	-3.17	1.71	-7.56	0.13	**44.67**	-1.41	-1.12	-6.49	-3.38
DeepSeek-V3	**81.22**	-4.02	-2.32	-7.07	-11.46	**48.05**	-8.78	-0.37	-5.98	-3.98	**50.04**	-3.69	-2.07	-7.46	-5.00
Claude-3.5	**76.71**	-8.66	-31.10	-2.44	-8.78	**41.22**	-6.71	0.24	-4.04	-3.44	**45.16**	-0.84	-1.70	-6.50	-3.08
GPT-4	**72.68**	-8.78	-3.66	-6.34	-12.56	**45.24**	-29.14	-28.65	-30.12	-29.26	**47.62**	-28.38	-26.94	-32.35	-28.67
DeepSeek-R1	**77.68**	-1.09	-1.58	-4.02	-4.39	**33.90**	-4.27	0.04	-10.12	-2.31	**31.42**	-2.14	-2.08	-9.70	-2.84
Pass@3 (%)															
CodeLlama	**40.49**	-4.64	-2.93	-1.16	-8.36	**15.73**	-3.17	-2.32	-3.53	-1.46	**14.40**	-2.30	-1.77	-3.30	-1.15
Qwen-2.5-Coder	**70.55**	-6.40	-2.44	-1.53	-7.14	**47.56**	-2.13	2.87	-4.76	0.51	**50.91**	-0.61	-0.66	-4.02	-2.46
DeepSeek-V3	**83.41**	-2.50	-2.62	-7.43	-11.40	**53.48**	-9.33	-0.31	-3.00	-2.41	**58.26**	-3.01	-1.20	-6.31	-3.80
Claude-3.5	**89.94**	-8.23	-26.34	-6.83	-9.45	**46.28**	-5.91	1.77	-4.94	-2.62	**49.48**	-0.68	-1.17	-4.97	-2.53
GPT-4	**82.68**	-10.85	-4.14	-8.11	-14.39	**54.02**	-29.81	-29.08	-30.97	-31.46	**56.71**	-28.11	-26.37	-33.44	-28.95
DeepSeek-R1	**86.83**	-1.46	-1.59	-3.66	-4.21	**44.08**	-6.34	0.01	-11.11	-2.74	**43.10**	-2.66	-3.09	-10.97	-4.03
Pass@5 (%)															
CodeLlama	**43.29**	-3.66	-2.44	-0.61	-8.53	**23.17**	-4.88	-1.83	-4.88	-2.44	**20.70**	-3.20	-2.65	-4.51	-1.17
Qwen-2.5-Coder	**70.73**	-6.10	-1.83	-1.22	-6.10	**48.78**	-1.83	3.66	-3.05	-0.36	**52.56**	-0.10	-0.41	-2.56	-2.05
DeepSeek-V3	**84.76**	-2.44	-3.66	-8.54	-12.20	**54.88**	-9.15	-0.40	-1.83	-1.51	**61.17**	-2.97	-1.03	-5.84	-3.69
Claude-3.5	**92.07**	-4.87	-22.56	-6.70	-8.53	**48.17**	-6.10	1.83	-4.83	-2.82	**50.72**	-0.42	-0.62	-2.84	-1.69
GPT-4	**86.59**	-12.81	-4.88	-9.76	-16.47	**57.31**	-29.87	-28.65	-30.48	-31.09	**60.04**	-27.36	-25.72	-33.20	-28.34
DeepSeek-R1	**90.24**	-2.44	-2.44	-3.04	-6.09	**48.78**	-6.71	-1.05	-11.74	-3.66	**47.38**	-2.20	-3.92	-10.90	-4.32

* The abbreviation Orig. stands for original requirement (*i.e.*, without ambiguity).
* The abbreviations Lex, Syn, Sem and Vag stand for lexical, syntactic, semantic and vagueness ambiguity, respectively.
* Δ = Pass@$k_{\text{ambiguous}}$ − Pass@k_{original}.

5.3 RQ2: Functional Consistency

We evaluate functional consistency by calculating the average conflict rate across five responses per model (intra-model) and across all responses from five models (inter-model).

As summarized in Table 5, ambiguous requirements substantially increase functional divergence in LLM-generated code. While clear requirements maintain relatively higher consistency, ambiguity nearly doubles the conflict rates for capable models on Orchid-HEval. Specifically, GPT-4 increases from 14.09% to 28.29%, and DeepSeek-V3 from 6.83% to 17.45%. This trend persists across ambiguity types; for example, lexical ambiguity alone increases Qwen-2.5-Coder's conflict rate on Orchid-BCB from 16.89% to 21.52%.

Notably, CodeLlama does not exhibit a similar increase, as its Pass@1 of only 6.22% limits the availability of correct outputs, making consistency comparison infeasible. Furthermore, ambiguity induces substantial divergence across models. On Orchid-BCB, the inter-model conflict rate reaches 57.28%, indicating a lack of consensus among LLMs when interpreting ambiguous requirements.

Figure 8 further illustrates the functional fragmentation introduced by ambiguity. The central green region represents the functionality derived from clear requirements, while the separated regions indicate that ambiguity leads to multiple incompatible functional interpretations. For instance, in Orchid-HEval Task #119 (Figure 8a), GPT-4 generates up to five distinct functional variants, resulting in implementations that diverge significantly from the intended behavior. Similar patterns are observed across other models, as shown in Figures 8b and 8c, confirming that ambiguity consistently undermines functional consistency.

> **Finding 2:** Ambiguous requirements lead LLMs to generate functionally inconsistent code. For instance, ambiguity nearly doubles GPT-4's intra-model conflict rate to 28.29% and increases the inter-model conflict rate from 51.04% to 57.28%.

5.4 RQ3: Identification and Resolution

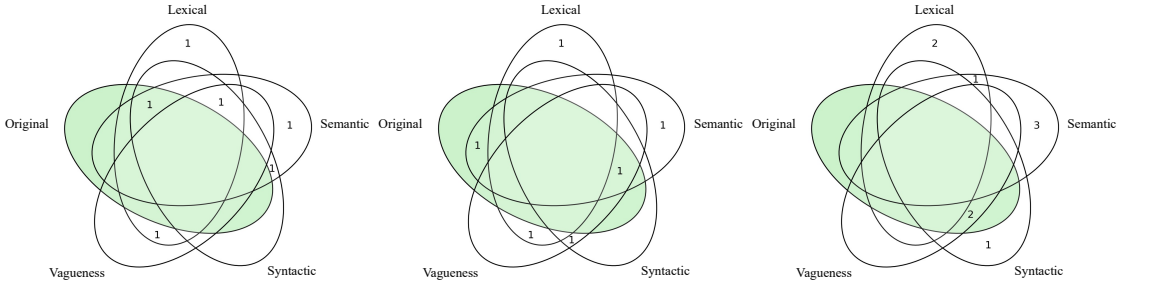
We evaluate each model on both the original clear requirements and their ambiguous variants using a fixed prompt that asks whether ambiguity is present and, if so, requests an explanation without generating code. GPT-4 (`gpt-4`) then compares each response with the corresponding reference annotation and assigns one of the four recognition levels defined above. The judge receives only the model response and reference annotation, without the identity of the evaluated model. The evaluated API models and the judge use a temperature of 0.0, while local models use greedy decoding. *Detection*, *Localization*, and *Tackling* are counted as positive ambiguity detections when computing precision and recall. We manually inspect a random subset of 50 responses and observe 96% agreement between the automatic and human judgments.

Figure 9 shows a representative example from Orchid-HEval #65, which involves circularly shifting the digits of an integer x. The phrase "shift the digits in a direction by shift" is inherently ambiguous due to the unspecified direction, leading to multiple possible interpretations and outputs. In this case, GPT-4 successfully detects and localizes the ambiguous segment but does not provide a concrete resolution. Accordingly, its response is categorized as successful localization only.

Table 5: Conflict Rate of LLMs on original and ambiguous requirements.

Models	Orchid-HEval (%)		Orchid-BCB (%)		Orchid-BCB-Expand (%)	
	Original	Ambiguous	Original	Ambiguous	Original	Ambiguous
CodeLlama	23.96	**38.61**	**50.12**	47.62	**48.24**	45.93
Qwen-2.5-Coder	5.67	**22.07**	16.89	**28.06**	18.49	**31.16**
DeepSeek-V3	6.83	**17.45**	17.07	**26.23**	21.73	**31.62**
Claude-3.5	16.83	**21.16**	12.68	**24.01**	13.10	**22.48**
GPT-4	14.09	**28.29**	22.80	**31.42**	24.70	**35.33**
Multi-models	30.60	**36.45**	51.04	**57.28**	54.30	**58.56**

* **Bold** indicates the higher conflict rate between Original and Ambiguous.



(a) GPT-4 responses for Orchid-HEval Task #119 (b) Claude-3.5 responses for Orchid-BCB Task #132 (c) DeepSeek-V3 responses for Orchid-BCB-Expand Task #908

Figure 8: Functional diversity of LLMs on Orchid.

Table 6 summarizes the evaluation results. While recall varies across models, precision remains consistently around 50%, indicating that clear requirements are frequently misclassified as ambiguous. Overall, all evaluated LLMs struggle to reliably distinguish ambiguity from complex but well-defined requirements, and tend to adopt a conservative strategy that favors over-detection. This results in a high false positive rate and consequently low precision.

Among the evaluated models, Claude-3.5 exhibits the most pronounced behavior. By adopting a highly sensitive detection strategy to minimize missed ambiguities, it achieves near-perfect recall (often exceeding 96%, and reaching 100% for semantic ambiguity). However, this comes at the expense of precision, which remains around 50%, indicating that a substantial portion of clear requirements are incorrectly classified as ambiguous.

> **Finding 3:** LLMs struggle to distinguish ambiguous requirements. They tend to regard most requirements as ambiguous (precision around 50%).

We further analyze the levels of LLMs' ambiguity recognition in Table 6. For each case where a model identifies ambiguity, its response is categorized into three progressive levels: detection (Det), localization (Loc), and tackling (Tac). Overall, while LLMs demonstrate reasonable capability in detecting ambiguous requirements, their ability to localize and tackle ambiguity remains limited.

On Orchid-BCB, this gap is evident across models. For example, Claude-3.5 detects 73.2% of syntactic ambiguities, yet achieves only 7.9% in localization and 17.1% in tackling. Similarly, GPT-4 detects 43.9% of vagueness cases, but attains a localization rate of 15.2% and a tackling rate of 21.3%. These results indicate that although ambiguity can often be identified, transforming detection into precise localization and actionable resolution remains challenging.

We further observe that recognition performance varies across ambiguity types. Lexical ambiguities are detected at 40.9%, syntactic ambiguities at 43.7%, semantic ambiguities at 24.7%, and vagueness at 38.8%, while localization rates are 11.1%, 13.7%, 12.5%, and 11.1%, respectively. In contrast, semantic ambiguities achieve the highest conditional tackling rate across model-dataset combinations, with 47.0% of recognized cases reaching the tackling level on average, suggesting that they are more amenable to resolution once identified than other ambiguity types.

> **Finding 4:** LLMs struggle to localize and tackle ambiguous requirements. Across evaluated models, localization is consistently poor, rarely exceeding 23%, while tackling remains below 64%.

6 Learned Lessons

Based on our findings, we summarize several key lessons on how to handle ambiguous requirements, with particular emphasis on stability, consistency, and ambiguity recognition.

1) High Performance Does Not Necessarily Translate to Stability Under Ambiguity. Even state-of-the-art models, such as GPT-4, exhibit notable performance degradation when exposed to ambiguous requirements, whereas some models with comparatively lower baseline performance, such as Qwen-2.5-Coder, demonstrate relatively stable behavior across such inputs. This suggests that leaderboard

Table 6: Evaluation of LLM capability of recognizing ambiguity in requirements.

Models	Orchid-HEval						Orchid-BCB						Orchid-BCB-Expand					
	Pre(%)	Rec(%)	Una(%)	Det(%)	Loc(%)	Tac(%)	Pre(%)	Rec(%)	Una(%)	Det(%)	Loc(%)	Tac(%)	Pre(%)	Rec(%)	Una(%)	Det(%)	Loc(%)	Tac(%)
Lexical Ambiguity																		
GPT-4	55.2	78.7	21.3	20.7	13.5	44.5	47.4	79.3	20.8	45.1	14.6	19.5	48.3	84.0	16.0	48.8	15.5	19.7
Claude-3.5	50.8	97.0	3.0	31.1	6.1	59.8	49.7	97.6	2.4	70.1	5.5	22.0	50.5	99.7	0.3	68.4	9.7	21.6
DeepSeek-V3	55.4	78.0	22.0	21.3	11.0	45.7	50.5	89.0	11.0	54.8	5.5	28.7	50.1	88.7	11.3	53.2	11.2	24.3
Qwen-2.5-Coder	51.8	87.8	12.2	28.6	17.7	41.5	55.9	89.6	10.4	56.7	12.8	20.1	51.8	87.8	12.2	52.1	16.5	19.2
CodeLlama	57.1	39.0	61.0	18.9	12.8	7.3	42.0	35.4	64.6	23.8	6.7	4.9	39.2	30.6	69.4	20.1	7.2	3.3
Syntactic Ambiguity																		
GPT-4	51.8	68.9	31.1	33.5	15.3	20.1	48.7	83.5	16.5	50.5	16.5	16.5	48.9	85.8	14.2	45.3	21.7	18.8
Claude-3.5	50.0	93.9	6.1	48.8	12.2	32.9	49.8	98.1	1.8	73.2	7.9	17.1	49.6	98.9	1.1	62.9	14.6	21.4
DeepSeek-V3	53.2	71.3	28.7	29.8	12.2	29.3	50.7	89.7	10.4	51.8	16.5	21.3	50.5	89.6	10.5	45.4	16.8	27.3
Qwen-2.5-Coder	49.6	80.5	19.5	47.6	15.2	17.7	54.7	85.4	14.6	55.5	15.9	14.0	51.6	86.6	13.4	49.6	20.9	16.1
CodeLlama	49.5	28.7	71.3	21.4	4.3	3.0	38.0	29.9	70.1	22.0	6.1	1.8	39.9	31.6	68.4	18.6	9.2	3.8
Semantic Ambiguity																		
GPT-4	51.4	67.7	32.3	15.9	21.3	30.5	50.0	87.8	12.2	24.4	12.8	50.6	50.0	89.8	10.2	25.1	16.9	47.8
Claude-3.5	50.8	96.9	3.0	29.9	17.1	50.0	50.3	100.0	0.0	32.9	6.1	61.0	50.1	99.7	0.3	36.8	7.8	55.1
DeepSeek-V3	50.9	65.2	34.8	15.2	11.0	39.0	51.9	93.9	6.1	24.4	6.7	62.8	51.7	94.6	5.4	22.2	9.3	63.1
Qwen-2.5-Coder	51.1	85.4	14.6	32.9	14.7	37.8	56.6	92.1	7.9	26.3	20.1	45.7	52.9	91.3	8.7	28.3	23.0	40.0
CodeLlama	50.5	29.9	70.1	20.1	4.3	5.5	40.3	32.9	67.1	21.3	7.3	4.3	39.3	30.8	69.2	15.5	9.7	5.6
Vagueness Ambiguity																		
GPT-4	57.1	85.4	14.6	28.6	15.9	40.9	47.8	80.5	19.6	43.9	15.2	21.3	48.7	85.4	14.7	42.8	17.8	24.7
Claude-3.5	51.3	98.8	1.2	28.7	7.9	62.2	50.3	100.0	0.0	65.2	4.3	30.5	50.0	99.3	0.7	56.5	6.8	36.0
DeepSeek-V3	58.5	88.4	11.6	26.2	8.5	53.7	50.3	88.4	11.6	51.8	4.9	31.7	50.8	90.6	9.4	42.3	11.3	37.0
Qwen-2.5-Coder	53.5	93.9	6.1	32.3	17.1	44.5	55.2	87.2	12.8	52.4	17.1	17.7	51.9	87.6	12.4	48.8	17.3	21.5
CodeLlama	49.5	28.7	71.3	16.5	6.1	6.1	42.5	36.0	64.0	23.8	7.9	4.3	41.5	33.7	66.3	22.7	8.0	3.0

* Darker blue cells indicate a higher percentage of responses for that category.

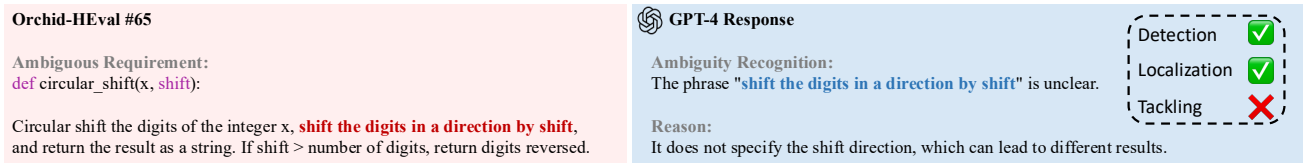


Figure 9: Example from Orchid-HEval #65 where GPT-4 recognizes and localizes ambiguity.

performance on well-formed benchmarks is insufficient to characterize model effectiveness in realistic software engineering scenarios, where requirements are often underspecified or ambiguous. For practitioners, this implies that model selection should incorporate targeted evaluation under project-specific ambiguity patterns rather than relying solely on aggregate benchmark rankings.

2) Ambiguity Harms Both Intra- and Inter-Model Consistency. Requirement ambiguity increases variability not only across different models but also across multiple outputs from the same model. Such inconsistencies reflect uncertainty in interpreting the requirements and can serve as an empirical signal of underlying ambiguity. In practice, developers can leverage this property by generating multiple candidate solutions or comparing outputs across models. Significant divergence in functional or logical behavior should be treated as a warning sign, prompting further clarification of the requirements before proceeding with implementation.

3) Sensitivity to Ambiguity Is Type-Dependent. LLMs exhibit uneven sensitivity to different categories of ambiguity. Some models are more affected by syntactic or vague expressions, while others show relatively minor performance variation across ambiguity types. This indicates that ambiguity should not be treated as a uniform phenomenon when evaluating code generation systems. Instead, fine-grained analysis is necessary to understand model behavior under different ambiguity conditions. These type-specific differences also have practical implications for requirements engineering. For development teams, identifying recurring ambiguity patterns in internal requirements (e.g., syntactic ambiguity or vagueness) can inform the design of guidelines and documentation practices that reduce ambiguity and improve the reliability of LLM-assisted development.

4) Ambiguity Recognition Remains a Limiting Factor. Across all evaluated models, the precision of ambiguity detection is limited, leading to frequent false positives. This constrains the models' ability to reliably assess whether a requirement is ambiguous. As a result, developers should not assume that LLM outputs are trustworthy indicators of requirement clarity. Instead, ambiguous or complex inputs should be treated as potentially misinterpreted, and clarification strategies (e.g., prompting the model to restate requirements, explain assumptions, or outline its intended solution) should be employed to validate outputs before integration into codebases.

7 Related Work

7.1 Code Generation Benchmarks

The evaluation of Large Language Models (LLMs) for code generation has evolved from controlled algorithmic tasks to more complex and realistic scenarios. Early benchmarks such as HumanEval [12] and MBPP [4] consist of manually curated programming problems with concise, well-defined requirements and deterministic expected outputs. These benchmarks primarily focus on assessing functional correctness under unambiguous specifications.

To improve evaluation diversity and realism, recent benchmarks including MultiPL-E [10], BigCodeBench [42], LiveCodeBench [26], and SWE-Bench [28] extend the scope to multi-language settings, dynamic execution environments, and repository-level tasks. These efforts introduce more complex programming scenarios and better approximate real-world development conditions.

Despite these advances, a key characteristic shared by existing benchmarks is that their requirements are intentionally designed to be unambiguous and deterministic. While this design facilitates consistent evaluation and reproducibility, it abstracts away the uncertainty inherent in natural language specifications. As a result, these benchmarks primarily evaluate model performance under idealized conditions where each input corresponds to a single intended interpretation. They do not capture how models behave when requirements admit multiple valid interpretations.

7.2 Ambiguity in Requirements Engineering

Ambiguity in Requirements Engineering refers to situations where a specification admits multiple valid interpretations [6, 36]. It is recognized as a major source of defects, misunderstandings, and inconsistencies in software development. Prior research has extensively investigated ambiguity from a human-centric perspective, aiming to improve requirement quality and support stakeholders.

A range of techniques has been proposed for ambiguity detection and resolution. These include methods combining NLP with fuzzy set theory to identify ambiguities and clarify vague requirement terms [2], rule-based approaches for identifying structural ambiguities [37], and machine learning methods leveraging contextual representations such as BERT to address ambiguity [14]. Additional work uses domain-specific word embeddings to identify cross-domain terminological ambiguity and knowledge graphs to detect pragmatic ambiguity [17, 18].

These approaches are designed for human-in-the-loop scenarios, where ambiguity can be resolved through clarification, negotiation, and iterative refinement [19]. However, LLM-based code generation operates under a different paradigm, in which the model must directly map a potentially ambiguous input to a single executable output without access to explicit clarification. This difference in operational assumptions limits the direct applicability of existing RE techniques and suggests that ambiguity should be re-examined in the context of automated code generation systems.

7.3 Ambiguity-Aware Code Generation

Recent work has begun to investigate how ambiguity affects LLM-based code generation. Existing efforts can be broadly categorized into benchmark-based evaluation and method-oriented approaches.

On the benchmark side, datasets such as AmbiQT [7] and HumanEvalComm [39] incorporate ambiguous or underspecified requirements to evaluate model robustness. These benchmarks consider scenarios involving multiple valid solutions, vague instructions, and implicit intent inference. However, AmbiQT focuses on schema-dependent ambiguity in text-to-SQL generation, while HumanEvalComm evaluates whether models ask clarification questions when requirements contain ambiguity, inconsistency, or incompleteness. Neither provides paired analysis of multiple linguistic ambiguity types in general-purpose function-level code generation.

On the methodological side, several approaches address ambiguous or unclear specifications. Interactive frameworks such as ClarifyGPT [32] and multi-agent systems [16, 27] introduce clarification mechanisms or collaborative reasoning processes to refine user intent. Other methods incorporate execution feedback, test cases, or iterative refinement strategies to improve code correctness.

Despite these efforts, it remains unclear how linguistic ambiguity types affect LLM behavior under matched task and test conditions. We address this gap by pairing each function-level task with four type-specific ambiguity variants and analyzing their effects on functional correctness, output consistency, and ambiguity handling.

8 Limitations

Our evaluation focuses on function-level code generation, which enables controlled comparisons but leaves ambiguity in broader software engineering contexts, such as repository-level tasks with cross-function dependencies and evolving requirements, for future investigation. Each benchmark variant contains one ambiguity type introduced at a single location to isolate its effect. Future work can extend Orchid to requirements containing multiple interacting ambiguities, incompleteness, or inconsistency.

9 Conclusion

This paper studies how requirement ambiguity affects LLM-based code generation. We introduce Orchid, a benchmark consisting of 1,304 function-level tasks with ambiguous requirements across four linguistic categories, and use it to systematically investigate model behavior under uncertain specifications. Our empirical results show that ambiguity consistently degrades generation performance and reduces functional consistency across model outputs. We further find that, although LLMs can often identify ambiguous requirements with relatively high recall, they exhibit limited precision and struggle to accurately localize and resolve the sources of ambiguity. Overall, our findings indicate that current LLM-based code generation systems are sensitive to requirement ambiguity and lack robustness in handling uncertain natural language specifications. These results highlight ambiguity as a critical factor in practical software development and motivate the need for ambiguity-aware approaches in future LLM-based software engineering systems.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant Nos. 62402183, 92582108, 62572192, 62372181). It is also supported by the Shanghai Special Program for Promoting High-Quality Industrial Development (Project No. 250401) and the Fundamental Research Funds for the Central Universities.

10 Data Availability Statement

The artifact for this study includes the Orchid benchmark (comprising 1,304 tasks and 5,216 requirements), the multi-agent ambiguity construction pipeline, and the raw experimental outputs from all evaluated LLMs. It is available on Figshare [40].

References

[1] Anthropic. 2024. Introducing Claude 3.5 Sonnet. https://www.anthropic.com/news/claude-3-5-sonnet.

[2] Mehdi Rajabi Asadabadi, Morteza Saberi, Ofer Zwikael, and Elizabeth Chang. 2020. Ambiguous Requirements: A Semi-Automated Approach to Identify and Clarify Ambiguity in Large-Scale Projects. *Computers & Industrial Engineering* 149 (2020), 106828. doi:10.1016/j.cie.2020.106828

[3] Ben Athiwaratkun, Sanjay Krishna Gouda, Zijian Wang, Xiaopeng Li, Yuchen Tian, Ming Tan, Wasi Uddin Ahmad, Shiqi Wang, Qing Sun, Mingyue Shang, et al. 2022. Multi-lingual evaluation of code generation models. *arXiv preprint arXiv:2210.14868* (2022). doi:10.48550/arXiv.2210.14868

[4] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021). doi:10.48550/arXiv.2108.07732

[5] Muneera Bano. 2015. Addressing the challenges of requirements ambiguity: A review of empirical literature. In *2015 IEEE fifth international workshop on empirical requirements engineering (EmpiRE)*. IEEE, 21–24. doi:10.1109/EMPIRE.2015.7431303

[6] Daniel M Berry and Erik Kamsties. 2004. Ambiguity in requirements specification. In *Perspectives on software requirements*. Springer, 7–44. doi:10.1007/978-1-4615-0465-8_2

[7] Adithya Bhaskar, Tushar Tomar, Ashutosh Sathe, and Sunita Sarawagi. 2023. Benchmarking and improving text-to-sql generation under ambiguity. *arXiv preprint arXiv:2310.13659* (2023). doi:10.48550/arXiv.2310.13659

[8] Frederik P Brooks. 1987. Essence and accidents of software engineering. *IEEE computer* 20, 4 (1987), 10–19. doi:10.1109/MC.1987.1663532

[9] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[10] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, et al. 2023. Multipl-e: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering* 49, 7 (2023), 3675–3691. doi:10.1109/TSE.2023.3267446

[11] Federico Cassano, Luisa Li, Akul Sethi, Noah Shinn, Abby Brennan-Jones, Jacob Ginesin, Edward Berman, George Chakhnashvili, Anton Lozhkov, Carolyn Jane Anderson, et al. 2023. Can it edit? evaluating the ability of large language models to follow code editing instructions. *arXiv preprint arXiv:2312.12450* (2023). doi:10.48550/arXiv.2312.12450

[12] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021). doi:10.48550/arXiv.2107.03374

[13] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2024. Evaluating large language models in class-level code generation. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13. doi:10.1145/3597503.3639219

[14] Saad Ezzini, Sallam Abualhaija, Chetan Arora, and Mehrdad Sabetzadeh. 2022. TAPHSIR: towards AnaPHoric ambiguity detection and ReSolution in requirements. In *Proceedings of the 30th ACM joint european software engineering conference and symposium on the foundations of software engineering*. 1677–1681. doi:10.1145/3540250.3558928

[15] Saad Ezzini, Sallam Abualhaija, Chetan Arora, Mehrdad Sabetzadeh, and Lionel C Briand. 2021. Using domain-specific corpora for improved handling of ambiguity in requirements. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1485–1497. doi:10.1109/ICSE43902.2021.00133

[16] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K Lahiri. 2024. Llm-based test-driven interactive code generation: User study and empirical evaluation. *IEEE Transactions on Software Engineering* (2024). doi:10.1109/TSE.2024.3428972

[17] Alessio Ferrari and Andrea Esuli. 2019. An NLP approach for cross-domain ambiguity detection in requirements engineering. *Automated Software Engineering* 26, 3 (2019), 559–598. doi:10.1007/s10515-019-00261-7

[18] Alessio Ferrari and Stefania Gnesi. 2012. Using Collective Intelligence to Detect Pragmatic Ambiguities. In *2012 20th IEEE International Requirements Engineering Conference (RE)*. IEEE, 191–200. doi:10.1109/RE.2012.6345803

[19] Jannik Fischbach, Julian Frattini, Daniel Mendez, Michael Unterkalmsteiner, Henning Femmer, and Andreas Vogelsang. 2021. How do practitioners interpret conditionals in requirements?. In *Product-Focused Software Process Improvement: 22nd International Conference, PROFES 2021, Turin, Italy, November 26, 2021, Proceedings 22*. Springer, 85–102. doi:10.1007/978-3-030-91452-3_6

[20] Emanuele Gentili and Davide Falessi. 2023. Characterizing Requirements Smells. In *International Conference on Product-Focused Software Process Improvement*. Springer, 387–398. doi:10.1007/978-3-031-49266-2_27

[21] Vincenzo Gervasi and Didar Zowghi. 2005. Reasoning about inconsistencies in natural language requirements. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 14, 3 (2005), 277–330. doi:10.1145/1072997.1072999

[22] Linyuan Gong, Sida Wang, Mostafa Elhoushi, and Alvin Cheung. 2024. Evaluation of llms on syntax-aware code fill-in-the-middle tasks. *arXiv preprint arXiv:2403.04814* (2024). doi:10.48550/arXiv.2403.04814

[23] Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I Wang. 2024. Cruxeval: A benchmark for code reasoning, understanding and execution. *arXiv preprint arXiv:2401.03065* (2024). doi:10.48550/arXiv.2401.03065

[24] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025). doi:10.48550/arXiv.2501.12948

[25] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024). doi:10.48550/arXiv.2409.12186

[26] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974* (2024). doi:10.48550/arXiv.2403.07974

[27] Haoxiang Jia, Robbie Morris, He Ye, Federica Sarro, and Sergey Mechtaev. 2025. Automated Repair of Ambiguous Problem Descriptions for LLM-Based Code Generation. *arXiv preprint arXiv:2505.07270* (2025). doi:10.48550/arXiv.2505.07270

[28] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770* (2023). doi:10.48550/arXiv.2310.06770

[29] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-tau Yih, Daniel Fried, Sida Wang, and Tao Yu. 2023. DS-1000: A natural and reliable benchmark for data science code generation. In *International Conference on Machine Learning*. PMLR, 18319–18345.

[30] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024). doi:10.48550/arXiv.2412.19437

[31] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems* 36 (2023), 21558–21572. doi:10.52202/075280-0943

[32] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binquan Zhang, ChenXue Wang, Shichao Liu, and Qing Wang. 2024. Clarifygpt: A framework for enhancing llm-based code generation via requirements clarification. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 2332–2354. doi:10.1145/3660810

[33] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474* (2022). doi:10.48550/arXiv.2203.13474

[34] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023). doi:10.48550/arXiv.2303.08774

[35] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023). doi:10.48550/arXiv.2308.12950

[36] Unnati S Shah and Devesh C Jinwala. 2015. Resolving ambiguities in natural language software requirements: a comprehensive survey. *ACM SIGSOFT Software Engineering Notes* 40, 5 (2015), 1–7. doi:10.1145/2815021.2815032

[37] KATO Toshiharu and Kazuhiko Tsuda. 2022. A method of ambiguity detection in requirement specifications by using a knowledge dictionary. *Procedia Computer Science* 207 (2022), 1482–1489. doi:10.1016/j.procs.2022.09.205

[38] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv preprint arXiv:2109.00859* (2021). doi:10.48550/arXiv.2109.00859

[39] Jie JW Wu and Fatemeh H Fard. 2024. Humanevalcomm: Benchmarking the communication competence of code generation for llms and llm agent. *arXiv preprint arXiv:2406.00215* (2024). doi:10.48550/arXiv.2406.00215

[40] Di Yang. 2026. Orchid. figshare. Dataset, Version 3. doi:10.6084/m9.figshare.32899583.v3

[41] Pengcheng Yin, Bowen Deng, Edgar Chen, Bogdan Vasilescu, and Graham Neubig. 2018. Learning to mine aligned code and natural language pairs from stack overflow. In *Proceedings of the 15th international conference on mining software repositories*. 476–486. doi:10.1145/3196398.3196408

[42] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. 2024. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877* (2024). doi:10.48550/arXiv.2406.15877